

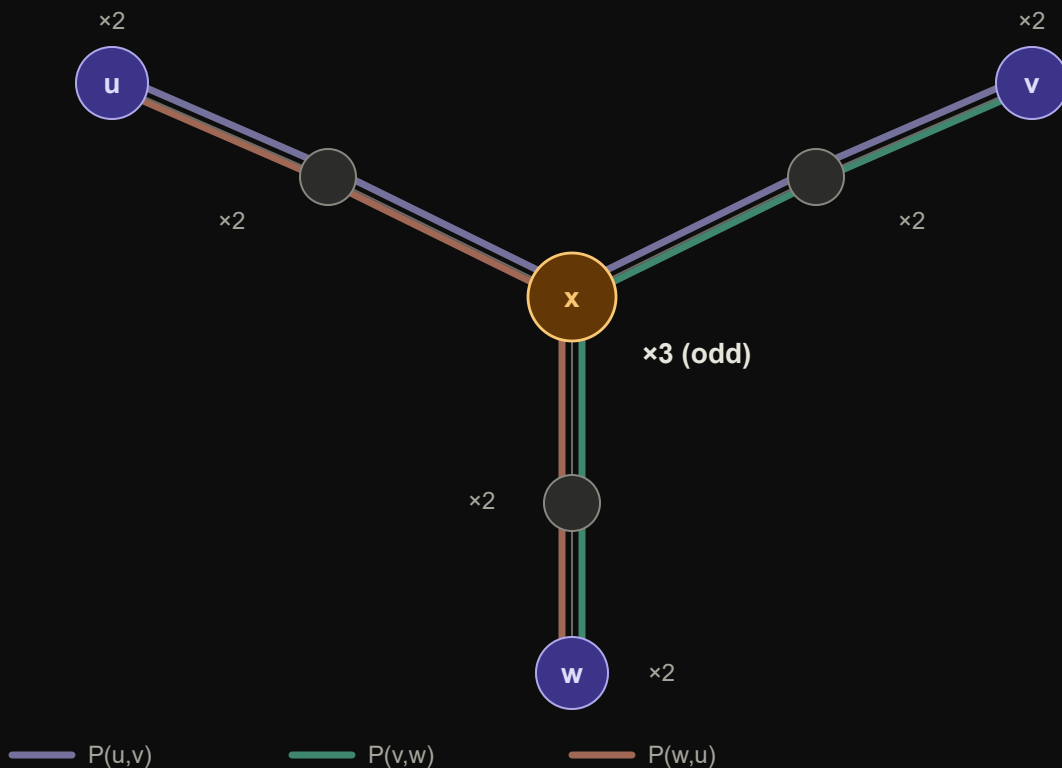
Good triplets in a tree — perfect square products

Problem. Tree on n vertices, value a_i on each. For a triplet $\{u, v, w\}$, is $p(u, v) \cdot p(v, w) \cdot p(w, u)$ a perfect square? Count all good triplets.

Key claim. For any triplet there is exactly one vertex c lying on all three pairwise paths (the "meeting point"). In the triple product, a_c appears **3 times (odd)**; every other vertex appears **0 or 2 times (even)**. Therefore the product is a perfect square **iff a_c is a perfect square**. No other vertex's value matters — not even u, v, w themselves (each endpoint sits on exactly 2 of the 3 paths).

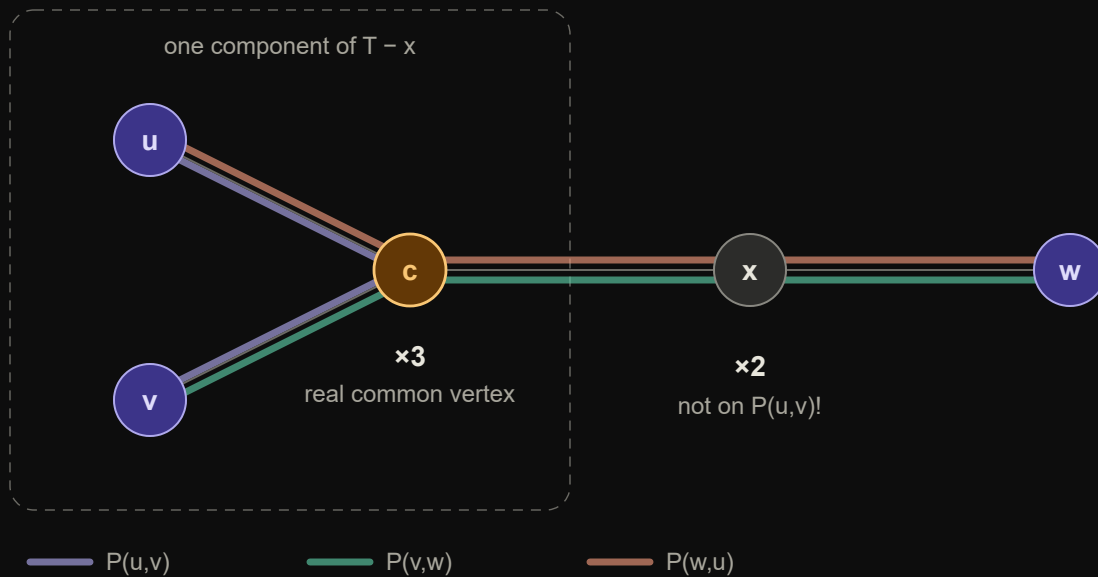
1. The forward picture — three different components

Here x is the meeting point, and u, v, w sit in three *different* branches (components of $T - x$). To travel between any two branches you are forced through x , so all three paths cross $x \rightarrow x$ is counted $\times 3$. Everything else on the paths is counted $\times 2$.



2. Why "three different components" is forced

Contrapositive: suppose two of the chosen vertices (say u and v) shared a component of $T - x$. Then $P(u,v)$ stays entirely inside that component and **never touches x** — so x is on only 2 of the 3 paths and cannot be the meeting point. The real meeting point is the branch vertex c inside that component (it is on all three paths, $\times 3$). The triplet is then credited to c 's tally, not x 's — nothing is lost or double-counted.



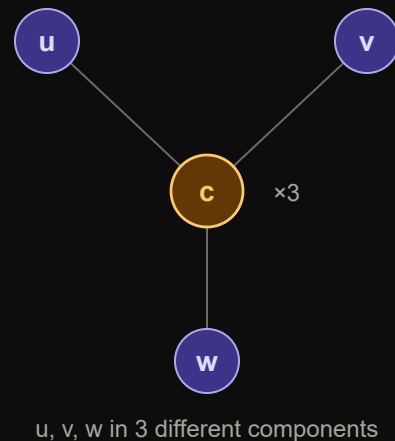
3. The two shapes of a good triplet around c

Fix a vertex c with a_c a perfect square. Every triplet whose meeting point is c has one of exactly two shapes:

Case 1 — c is chosen



Case 2 — c is not chosen



Case 1 (one of the triplet lies on the path between the other two, so it is c itself): the other two must come from two different components of $T - c$, otherwise their path skips c . This gives the $\sum s_i \cdot s_j$ term.

Case 2 (none of the three lies on a path between the other two): the meeting point is a fourth vertex c , and all three fall into three distinct components of $T - c$. This gives the $\sum s_i \cdot s_j \cdot s_l$ term.

Subtle point discussed. If v does not lie on $P(u,w)$, walk from v toward $P(u,w)$; the first vertex hit is c . If c happens to equal u or w , you're back in Case 1 with that vertex in the middle role. Otherwise c is interior to $P(u,w)$ — Case 2 — and removing c separates all three: v in its own branch, u and w on opposite sides of c along $P(u,w)$.

4. The $O(n)$ count

Root the tree, compute subtree sizes. For each vertex c , the components of $T - c$ are its child subtrees plus (if c isn't the root) the parent side of size $n - sz[c]$. If a_c is a perfect square, scan the component sizes once:

```
sum = pairs = triples = 0
for each component size s:
    triples += s * pairs
    pairs   += s * sum
    sum     += s
answer += pairs + triples
```

Each triplet has exactly one meeting point, so summing over all square-valued c counts every good triplet exactly once. Total time: **$O(n)$** per test case.